
Geometrically Consistent Generalizable Splatting

Abstract

Gaussian splatting has emerged as the preferred 3D scene representation due to its incredible speed and accuracy in novel view generation. Various attempts have thus been made in adapting multi-view structure prediction networks to predict per pixel 3D Gaussians from images directly. However, most of the existing work in this domain has focused on enhancing existing depth self-supervised depth prediction networks to estimate additional parameters for 3D Gaussians – orientations, scale, opacity and appearance. This paper shows that view-synthesis loss is insufficient to recover geometrically meaningful splats in this simple manner. We systematically analyse and address the inherent ambiguities in learning 3D Gaussian splats with self-supervision to learn pose-free generalisable splatting. The proposed approach provides state of the art (i) geometrically consistent reconstructions, (ii) relative pose between images and (iii) novel-view synthesis on RealState10K and ACID datasets. We also showcase zero-shot capabilities of the proposed generalizable splatting on ScanNet, where our method substantially outperforms the prior art in recovering geometry and relative pose as well.

1 Introduction

Gaussian splatting [5] has revolutionised 3D structure and appearance modelling from multiple images in the recent past. Departing from traditional depth or point cloud representations for the scene structure 3D Gaussians implicitly model surface reflections and environment lighting to encode view-dependent scene appearance, are memory-efficient compared to explicit volumetric alternatives and facilitate rendering of the scene from arbitrary viewpoints in a fraction of a second, utilising modern GPUs. Owing to the success of 3DGS, a significant effort has been put into improving the 3DGS to work with few-views [], in the presence of dynamically moving objects [] and in incorporating object semantics into 3D reconstructions []. Real-time simultaneous localisation and mapping approaches have been adapting splats as the inherent scene representation []. Splats are also used for generation of geometrically consistent images and video sequences []. 3D Gaussian splats have become the prevalent choice for representing scenes in this era.

The deep learning revolution of the last decade has drastically influenced geometric inference from one or more images. While earlier works in this direction focused on learning neural networks that map image(s) to their depth-maps provided by range sensors [?], recent works have explored reconstructing a registered set of per-pixel point clouds for multiple images to provide state-of-the-art relative pose and structure of the scene[?]. It has also been shown that these feed-forward neural networks to predict geometry from image(s) can be trained without depth sensors, in a self-supervised manner, by minimising view synthesis loss [? ? ? ? ?]. Structure prediction from single or a few images has been used as the optimisation-free building block in high-fidelity tracking and mapping systems []. Learning-assisted recovery of scene geometry has truly become mainstream.

It is natural then to marry the incredible capabilities of modern deep neural networks in learning useful geometric priors with the advantages of using 3D Gaussians as the scene representation. Various laudable attempts have been made recently in training neural networks to predict 3D splats directly from images [? ? ? ? ?]. In the literature, these methods are usually dubbed as Generalizable Gaussian Splatting methods.

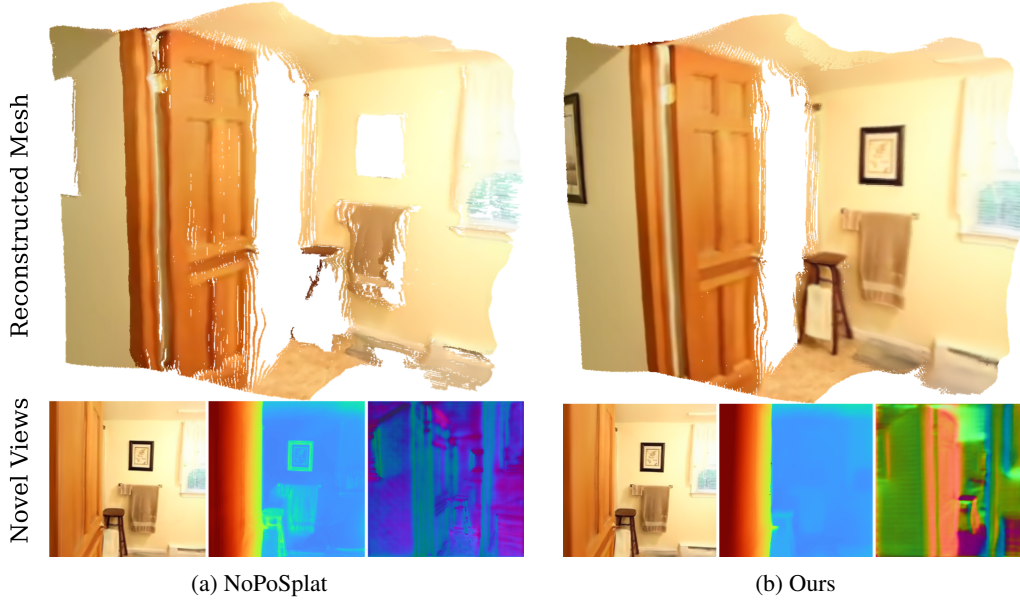


Figure 1: We reconstruct the mesh of the scene (1st row) from the splats using truncated signed distance function (TSDF) to fuse the rendered novel-view RGB images and depthmaps (2nd row). While NoPoSplat produces reasonably good appearance quality in the rendered RGB, it fails to learn accurate geometry, resulting in multiple holes in the reconstructed mesh due to the inconsistencies across the depthmaps during TSDF fusion. **Ideally need MVsplat results as well. If you get time ShinFang can you explain how these are generated in a para somewhere? Best done by you!**

Most Generalizable Gaussian Splatting methods focus on adapting well-studied structure prediction networks that estimate the means of the Gaussian. Generally, these networks consist of encoders for one or multiple images and decoders to predict Gaussian means. These means are either encoded in the form of per-pixel depths or point clouds, adapting two independent well-studied streams of works for two-view structure prediction via neural networks. Additional decoders are often appended directly to predict the remaining Gaussian parameters (orientation, scale, opacities and view-dependent colour) without much foresight in these methods. All or a partial set of encoders and decoders are learned by usually minimising view-synthesis loss on a few target views by these generalizable splatting methods. This prevalent setup used by Generalizable Gaussian Splatting methods overlooks several key issues that are inherited from the underlying 3DGS optimisation:

- 3D Splats are grossly over-parametrised compared to depth-maps or 3D point-clouds. Successful estimation of these 3D splats usually requires a large number of densely sampled viewpoints. 3DGS with few-views is an active field of research, with several solutions relying on heavy regularisation []. These developments are ignored in building a successful generalizable Gaussian splatting technique.
- Unlike per-pixel depths or 3D locations, which are uniquely defined for a scene (up to scale), several 3D Gaussian configurations can render input images equally well. Lacking a unique definition, it is unclear how to learn optimal 3D Gaussians even when scene structure is available for supervision in the form of depth maps.
- Successful Gaussian splatting relies on multiple non-differentiable heuristics (such as pruning, splitting, and duplication of Gaussian). Throughout the optimisation, many 3D Gaussians die (reach close to zero opacity), and many new Gaussians are introduced. In the absence of differentiable approximations of these heuristics, existing generalizable splatting approaches only rely on stochastic gradient descent optimisation of a fixed number of Gaussians. i.e. these approaches assume that per-pixel 3D Gaussians will remain perpetually alive during training without any substantial reason.

As a result, the current generalizable Gaussian splatting method often converges to geometrically degenerate scenarios. This is spatially true for Gaussian parameters other than their means – which

are relatively stable thanks to adaptation of well well-studied single-view depth estimator model initialisation and architectures.

Figure 2 showcases that current generalizable splatting frameworks fail to learn meaningful opacities, orientations, or scales of the 3D Gaussians when trained with only self supervised loss. We can see non-plausible 3D gaussian orientations (in the form of normals) as well as unjustifiable elongation of the 3D Gaussian (scales) from both dominant paradigms of generalizable splitting when self supervision is deployed. We show that these artefacts are due to overparameterization of geometry in the form of splats, which require structural consistency priors to make the self-supervised learning framework viable. Once these priors are introduced, the learned Gaussians from the proposed method exhibit various geometrically sensible patterns. Briefly, one can see that the proposed method reconstructs a very low number of splats with small opacities (near dead Gaussians) compared to baselines, can provide correct surface normals directly from the predicted Gaussian orientations, and produces Gaussian shapes elongated perpendicular to the normal edges and not along the normal edges. We will extensively discuss the geometric inconsistency present in the current Generalizable 3DGS results and how the proposed method alleviates these inconsistencies in section ???. The two approaches we picked for the visualisation of predicted 3D Gaussians broadly represent separate underlying representations for encoding Gaussian means, namely (i) per-pixel depth maps or (ii) per-pixel 3D locations aligned in a single frame of reference. However, to our knowledge, all self-supervised Generalizable 3DGS methods suffer from the same limitations as they inherit representation and loss function from [2, 9].

In this work, we systematically aim to define what the ideal configuration for the geometrically consistent Gaussian should look like and propose appropriate priors to assist Generalized Gaussian Splatting. To that end we opt to work on the recently published NoPo-Splat [9] as our baseline and enhance it. We select No-PoSplat as the baseline because, in addition to its state of performance on multiple geometric tasks, it is a self-supervised approach that does not require ground-truth depth maps, unlike some of the other generalizable splatting adaptations [?]. Additionally, utilising DUST3R [8] framework, the approach is one of the few to provide generalizable splatting from a pair of images without requiring the relative pose of these images.

We address ambiguities in learning over-parametrised 3D Gaussian splats by adding suitable regularisations to the traditionally used view-synthesis loss. Our main observations are: (i) defining the ideal 3D Gaussian orientations to be dominant normals of the scene surfaces (as advocated in [?]) helps resolve structural ambiguities and learning of the Gaussian orientations. We do that by integrating rasteriser from 2DGS framework [?], (ii) when relying on view synthesis loss, ensuring that the 3D Gaussians are pixel-aligned is extremely important. Particularly pose-free methods that use PnP for relative pose estimation between given images require this pixel alignment of Gaussians in ensuring accurate camera pose estimation as well as geometric structure prediction (iii) Jointly learning pixel-aligned normals and 3D location is challenging when self-supervision is used. Often, the optimisation gets stuck in a local minimum with degenerate planar reconstruction despite using existing priors used in [? ?]. Relying on depth-derived pseudo-normals for orientation supervision provides geometric accuracy.

We trained our generalizable splatting networks on RealState10k dataset [10] adapting pose-free approach of NoPoSplat [9] with appropriate geometric consistency priors. Our method outperform prior art on the tasks of novel-view synthesis, provides a plausible scene structure that can be rendered directly to provide depths from arbitrary views that current methods are incapable of. It also forms a new state of the art in relative pose estimation from two images surpassing existing approaches that include the task specific methods such as RoMa [4] by a large margin. We also showcase generalisation capability of our trained models on Scan-net [3] and ACID [6], where our Re10k trained models outperform all existing pose free generalizable splatting methods in relative-pose estimation and reach the performance of in-domain task specific pose estimators. Our approach provides the most accurate 3D reconstructions amongst all generalizable splatting approaches in the zero-shot testing setup on Scan-net datasets, including methods that use ground-truth relative pose to infer depth from two views. To the best of our knowledge, it is the first generalizable splatting approach to analyse and evaluate the veracity and meaningfulness of Gaussian Splatting parameters 3D gaussian orientations and elongations. We believe that the presented analysis pave the way forward for future research training direct 3D Gaussians from images in both depth supervised and self supervised manner.

2 Method

In this section, we introduce our generalizable splitting network, training procedure, and loss function we use to address the ill-posed nature of self-supervised learning in predicting geometrically consistent Gaussian splats from a set of images.

Architecture: Our approach adapts the N-view pose-free generalisable splitting architecture proposed by [9] and consists of three main modules: a transformer-based image encoder, two cross-attention-based asymmetric multi-view feature aggregators and a set of dense prediction transformer decoders to predict parameters of per-pixel Gaussian splats. Specifically, the image encoder transforms RGB image patches into a sequence of tokens and are augmented with the intrinsic parameters of the camera. These pre-image token sequences are processed by a ViT encoder independently. The resulting per-image features are subsequently fused via two sets of feature aggregators, each comprising of multi-head cross-attention layers, effectively aggregating information across multiple views. These transformer-based networks provide features for the prediction of Gaussian splats for the reference (first) and subsequent images, respectively. Parameters of the Gaussians are estimated by two separate DPT prediction heads: the first head directly infers Gaussian centres for input image pixels, the second head predicts the rest of the Gaussian parameters and combines fine image-level features for accurate appearance estimation.

Following [8], we use two asymmetric feature aggregators to provide the Gaussian centres for pixels of each frame to be aligned with the first image of the sequence. As all the recovered Gaussians are aligned, the union of them can be directly rendered to an arbitrary viewpoint without requiring any warping. Notably, both image tokeniser and feature aggregator employ Vision Transformer (ViT) backbones exclusively, without specialised architectures such as epipolar attention or multi-view cost-volumes. This geometry-free architecture does not require relative camera poses during training and inference and has been shown to perform on par or better than the generalisable splitting methods deploying these specialised architectures.

Notations and Problem Definition Assuming we are given with a set of $\mathcal{T}$ context images $\mathcal{I} = \{\mathbf{I}^i \in \mathbb{R}^{H \times W \times 3}\}_{i=1:\mathcal{T}}$, each with known camera intrinsics forming the set $\mathcal{K} = \{\mathbf{K}^i \in \mathbb{R}^{3 \times 3}\}_{i=1:\mathcal{T}}$, capturing a rigid scene. Our objective is to learn a feed-forward neural network with parameters θ that maps these images to a set of 3D Gaussian splats, one splat per pixel for all the input image such that:

$$f_{\theta}(\mathcal{I}, \mathcal{K}) = \left\{ \mathcal{G}_i^j := (\boldsymbol{\mu}_j^i, \alpha_j^i, \mathbf{q}_j^i, \mathbf{s}_j^i, \mathbf{c}_j^i) \right\}_{i=1:\mathcal{T}, j=1:H \times W} \quad (1)$$

where $\mathcal{G}_j^i$ is the 3D Gaussian corresponding to a pixel i in image j , characterized by its: 3D center $\boldsymbol{\mu}_j^i \in \mathbb{R}^3$; orientation represented by a unit quaternion vector $\mathbf{q} \in \mathbb{R}^4$; and two scale parameters $\mathbf{s} \in \mathbb{R}^2$ defining the elongation of 3D Gaussian splat; opacity $\alpha \in \mathbb{R}$; and appearance encoded in the form of coefficients of the spherical harmonic function of order d , $\mathbf{c} \in \mathbb{R}^d$. In this work, we advocate the use of per-pixel 2D Gaussians to represent the scene instead of the standard 3D Gaussian splats. Following [?], we assume that estimated Gaussians are aligned with the scene surface and its elongation perpendicular to the local surface normal is zero. We show through extensive evaluations how this choice help Generalizable Gaussian splitting.

Both, Gaussian centre $\boldsymbol{\mu}_j^i$ and orientations $\mathbf{q}_j^i$ are defined in the image coordinates of the first image from the set. Given these predicted $M \times N \times \mathcal{T}$ Gaussians¹, novel views $C^f \forall f \in \{1, \dots, \mathcal{F}\}$ of the scene can be rendered from viewpoints defined by the projection matrix $P^f = \mathbf{K}^f[\mathbf{R}^f|\mathbf{T}^f]$ to be matched with observed images I^f s. Like [9], we propose to minimise the view synthesis loss from the predicted 2D Gaussian splats:

$$L_{synthesis} = \sum_{i=1}^{\mathcal{F}} \mathcal{L}(I^f, C^f); \text{ where, } C^f(u, v) = \sum_{k=1}^K c_k \alpha_k \mathcal{G}_{2D}(u, v) \prod_{j=1}^{k-1} (1 - \alpha_j \mathcal{G}_{2D}(u, v)) \quad (2)$$

where $\mathcal{L}(a, b)$ represents the absolute sum of colour differences between observed and rendered images combined with the LPIPS error. $C^f(u, v)$ is the color of the pixel u, v in image C^f rendered

¹Like most other methods for generalizable splitting, one can predict more than one splat per pixel in our framework as well but we restrict our experiments to one splat per pixel only.

by blending K ordered 2D Gaussians using the 2DGS rasterizer. $\mathcal{G}_{2D}$ is the projection of the 2D Gaussians covariance matrix Σ_{2D} on the image plane of image i (See [?] for details).

As shown in section 4, this view synthesis loss is proven to be insufficient for learning geometrically meaningful splats. We propose to minimise two additional regularisation losses: (i) Depth-surface normal consistency term $\mathcal{L}_{normal}$ aligns 2D splat orientations with the rendered depth, and a grid alignment loss $\mathcal{L}_{grid}$ to ensure that the estimated splats are aligned with the pixels of the provided images. These two losses, alongside the view synthesis loss, form our training objective:

$$\mathcal{L} = \mathcal{L}_{synthesis} + \lambda_o \mathcal{L}_{orient} + \lambda_a \mathcal{L}_{align}, \quad (3)$$

where λ_o and λ_a are weighting factors balancing the influence of each regularization. The motivation, formulation and impact of the regularisation term have been discussed in the following sections.

2.1 Learning Gaussian Orientations.

Both pose-free and pose-aware generalised Gaussian splitting approaches fail in learning meaningful orientations of the splats. To provide geometric meaning for the splat orientations, we propose aligning them with the dominant surface normals of the scene they belong to. To that end, we follow the setup in [?] to estimate only two non-zero splat scales— instead of the usual three — and enforce the elongation of splats in the direction of surface normal to be zero. In particular, we model the rank-deficient Gaussian covariance matrix with its null space corresponding to the surface normal vector.

Swapping 3D splats to 2D surface elements reduces the over-parameterisation to an extent; however, the view synthesis loss of (??) does not give a strong enough supervision signal for orientations. The most natural way to supervise Gaussian orientation is to use surface normal regularisation from [?]. Authors propose to render 2D splat orientations in the form of a normal map with alpha blending and force these normals to be consistent with the rendered depth for a training view. This consistency term aligns 2D splats that contribute to the rendering of a single pixel to be perpendicular to the pixel’s rendered normal. While such prior has worked well in [?] and its variants used for few-shot Gaussian splitting techniques[], we observe that it can not be used naively in the context of generalisable splitting. Adding such regularisation for joint splat position and orientation estimation locks the generalizable splitting network into a local minimum, producing near-planar scenes. Detaching the rendered depth from the computation graph helps fix this problem. This setup can be seen as using depth rendering during optimisation to generate pseudo labels for orientations of 2D splats and produce reasonable splat orientation and locations. It should be noted that the successful training of the Gaussian orientation with such normal loss required L_{grid} . The results the [?] inspired loss for learning orientation are discussed in the supplementary material.

However, we opt to use a rather simple alternative to supervise the splat orientations in our approach. Assuming that the estimated splats $\mathcal{S}_i^j$ are aligned with the 2D image pixel $j = (u, v)$ in image i , we derive the local surface normal for this splat using the relative 3D locations of its local neighbourhood in image i . We propose to minimise the following normal loss:

$$L_{orient} = \frac{1}{\mathcal{T}(M-2)(N-2)} \sum_{i=1}^{\mathcal{T}} \sum_{u=2}^{M-1} \sum_{v=2}^{N-1} \|1 - \langle N_{u,v}^i, \hat{N}_{u,v}^i \rangle\|_{\epsilon} \quad (4)$$

$$\hat{N}_{u,v}^i = \|(\mu_{(u+1,v)}^i - \mu_{(u-1,v)}^i) \times (\mu_{(u,v+1)}^i - \mu_{(u,v-1)}^i)\|_* \quad (5)$$

where, $N_{u,v}^i$ is the null space of the Gaussian covariance matrix defining the normal to the plane defined by 2D splat corresponding to pixel (u, v) in image i , $\mu_{(u,v)}^i$ is the estimated 3D location of this splat’s centre, $\langle ., . \rangle$ is the dot product or two vectors, $\|\cdot\|_{\epsilon}$ is Huber loss (implemented as SmoothL1Loss in pytorch) and $\|\cdot\|_*$ represents vector normalization. Here we ignore the normal computation at the corner of the image and use central differences in computing local structure derivatives, however alternative padding and normal approximations can be used.

Note that, unlike the loss used in 2DGS [?] which involves rasterization of normals as well as depth to enforce consistency, the L_{orient} here does not involve any rasterisation. It is motivated by the simple observation that while depth supervised frameworks like DUST3R [8] predict registered 3D

point clouds corresponding to the pixels of input images, these point clouds are directly supervised from back-projected (and warped) depthmaps. By extension, if the ground truth depths were available for learning 3D/2D splats, the orientations can be directly supervised by the (warped) normal maps as well. This simple but effective way to directly supervise surface normals provides best generalizable splitting results under self-supervision. However, the same supervision can be used to improve orientation estimation in depth supervised splat learning frameworks such as [?] as well.

2.2 Pixel aligned Gaussian splats:

Although the first generalizable splitting approach worked in a pose-aware setup and adapted two-view depth predictor architectures, they by construction were designed to provide each 3D Gaussian to lie on a specific viewing ray. Pose free approaches aimed at directly estimating the 3D Gaussian locations as output of a DPT decoder head. While this alleviated the need for warping 3D Gaussian splats with known cameras, the parametrization rendered the structure estimation problem ill-posed especially under self supervised setup. Specifically, unlike depth supervision used in DUST3R [?] that learns a structural bias that the 3D point cloud projects to regular grid in the input image, view synthesis loss does not provide any such bias. 3D Gaussians thus can move freely to geometrically degenerate configurations hampering both structure and relative pose estimation of pose free methods. We enforce that the 3D Gaussians corresponding to a pixel of the input frame should lie on the corresponding viewing ray. To that end, we project centres $\mu_{u,v}^i$'s of reconstructed 3D Gaussians with known camera location and intrinsic matrix to match the location of pixels forming regular grids. We minimize the following loss:

$$L_{align} = \frac{1}{\sum_{i,u,v} \mathcal{M}_{u,v}^i} \sum_{i=1}^{\mathcal{T}} \sum_{u=1}^M \sum_{v=1}^N \mathcal{M}_{u,v}^i \| [u, v]^T - \Pi(K^i [R^i | T^i] \mu_{u,v}^i) \|_2^2; \quad (6)$$

where $\mathcal{M}_{u,v}^i$ is an indicator function defining splats getting projected inside the image and having positive depths and $([X, Y, Z]^T) = [X/Z, Y/Z]^T$ is the projection function. **Why is any of this needed again?? If a splat projects outside the image, the further it goes out, the more you should penalise it, right? same for positive depth vs negative depth, while negative depth makes less sense, you can still minimise the loss above??** The proposed loss plays a crucial role in PnP based relative pose estimation as well as accurate structure estimation, as seen in the results.

3 Analysis of figure 1

It is evident in Figure 2 that none of the existing Generalizable 3DGS approaches provide meaningful orientations or scales for the estimated Gaussians. In particular, we visualize the normal of the dominant plane of 3D Gaussians that corresponds to the direction in which the Gaussians is minimally elongated. This minimal elongation is labelled as scale 3 in the figure and quantifies the uncertainty in localising the scene surface around this Gaussian. The estimated normals from the existing Generalizable 3DGS methods lack any geometric meaning whereas the proposed method can provide meaningful normal maps conforming to plausible scene geometry. For example, notice multiple planes confirming to Manhattan prior in the scene in Figure ?? and the corresponding normal predictions.

The existing Generalised 3DGS methods does not provide geometrically meaningful estimates of the shape (scales) to the Gaussians either. One can observe clear over-fitting to view synthesis loss – which is highly sensitive to the location of image edges – in apparent image texture in the scales. Higher values in scale 3 suggest billboard-like elongated Gaussians perpendicular to the scene surface that existing Generalizable 3DGS approaches produce. On the other hand, our approach uses 2DGS rasterizer by construction, forcing this scale to be 0, eliminating this billboard-like Gaussians. Not only does the proposed approach localize scene surfaces, but the dominant structural edges are clearly visible in the estimated scales. For example, look at the interesting case of the edge of the wall at the intersection of two corridors ???. You can see that the Gaussians estimated by the proposed methods are very skewed in one direction than the other (one scale is very large and another very small). This is expected out of geometrically consistent Gaussians to have a very narrow extent perpendicular to geometric edges, whereas an arbitrarily large extent along the edge. You can also see clearly that more planar areas have more homogeneously elongated Gaussians. Whereas not as no clear geometric edge or homogeneity in the size and shape of Gaussians can be found in the existing approaches.

We can also see the opacities of the reconstructed Gaussians. Large regions of the image have very low opacities, pointing to many near-dead Gaussians estimated by the existing Generalizable 3dgs methods. These low opacity Gaussians are often found in the regions of shadows, light sources and reflection, suggesting that instead of modelling these variations with spherical harmonics as one would expect, the view-dependent colour is been modelled by changing opacities. In contrast, it can be seen that the proposed method consists of less number of near-dead Gaussians and low-opacity regions corresponding to image edges and shadows.

4 Experiments

4.1 Experimental Setup and Details

Datasets: To evaluate geometry and novel-view synthesis performance, we follow common practices in generalizable splatting methods [1, 2, 9] by training our model on the large-scale RealEstate10K [10] (RE10k) dataset. We use standard data splits on Re10K dataset and evaluate novel-view synthesis and pose estimation performance on the dataset for quantitative comparisons. ACID [6] and scannet [3] are used for evaluating the generalization capability of the our trained network on both relative pose and geometry estimation, where applicable.

RE10k comprises real estate videos (predominantly indoor scenes) downloaded from YouTube, and is divided into 67,477 training scenes and 7,289 testing scenes following the same train-test split as in previous works [1, 2]. ACID contains mainly aerial nature scenes captured by drones. For both datasets, camera poses were computed using COLMAP [7]. In contrast, ScanNet [3] consists of RGB-D captures of indoor scenes with different camera motions and scene characteristics compared to the other two datasets. We use depths and relative camera poses available in two test splits: ScanNet-V1 – comprising of 2000 image pairs for generalised splitting and ScanNet-1500 – comprising 1500 test image pairs.

Evaluation Metrics: We evaluate novel-view synthesis, the relative pose between the input images and the reconstructed geometry, where applicable, to benchmark our generalizable splatting network. Ours is a pose-free approach where two images alongside the intrinsic camera parameters are used to generate splats. Camera coordinate frame for the first image is used as the de-facto coordinate frame.

Novel views are synthesised by rendering all reconstructed splats for pose-free methods, given the fixed target pose w.r.t the first input image. For methods that estimate splat locations in the form of depth maps, an additional warping step with ground truth relative camera pose is used to generate set of splats aligned with the coordinate frame of the first image. Note that we do not opt for optimising camera pose for optimal target view synthesis. While many NeRF and splats method reports view synthesis errors by target image pose optimisation, it is well known that geometric inconsistency can be hidden by such pose optimisation easily. Moreover, we opt not to peak into the answers in synthesising novel views. We report PSNR, SSIM and LPIPS to quantify view synthesis performance.

Relative pose estimation is evaluated by computing the area under the cumulative pose error curve with three different thresholds. We report results for a simple PnP + RANSAC algorithm for our method and the NoPoSplat[9]. NoPosplat proposes a gradient descent-based relative pose refinement scheme in which the predicted Gaussian Splats are rendered to generate optimal input image pairs. Pose Jacobians from [] are used for this refinement for a fixed number of iterations. Although such a pose optimisation is not readily usable with 2D Gaussian splats. We convert our estimated 2D Gaussians to their 3D counterparts by augmenting a small non-zero scale to use the rasterizer and Jacobians as are used in NoPosplat to facilitate a comparison with the relative pose estimation results reported in [9].

Geometric veracity of the estimated 2D/3D Gaussian splats is the key focus of this work. Traditionally, the geometry predicted by feedforward neural networks is evaluated by measuring the depth errors for the input views. However, input depths do not capture the interpolation capability of splats, and are insensitive to the opacity, orientation, and scale parameters. We propose a more holistic evaluation of the scene structure by rendering multiple virtual depth maps and reporting the average Absolute Relative Error and depth rendering accuracy. As we do not focus on extrapolating the scene beyond the given input viewing frustum, we keep the same mechanism that is used in view-synthesis evaluation by NoPosplat for sampling the virtual view for depth evaluation. The virtual views are reconstructed using the ground truth relative pose with respect to the first input

frame. This puts pose-free methods at a severe disadvantage, as small alignment errors can result into large depth errors. A more accurate evaluation of reconstruction can be devised by rendering a large number of virtual depth maps from the generated Gaussian splats and fusing them back to create scene’s truncated signed distance field. 3D surface from the TSDF can be extracted using the marching cube algorithm to be compared with the ground truth, where available. We use such virtual-depth-rendering $\rightarrow$ TSDF fusion $\rightarrow$ surface extraction for visual comparison facilitated in the figure ?? . An appropriate distance measure, such as Chamfer distance between ground truth and generated mesh, is used as a good measure for pose error normalised surface reconstruction accuracy in the supplementary material.

Implementation details. All of our generalizable splat predictors are trained using the setup described in NoPoSplat [9]. Training is performed on a cluster of 24 A100 (40 GB) GPUs with a per-GPU batch size of 6 (total batch size 144), while all evaluations in Section 4 are run on a single A6000 GPU. Models are trained on the RE10k training split for 18751 iterations using 256×256 images and four rendered virtual views to minimise the view-synthesis loss. Regularisation weights are set to $\lambda_o = 0.05$ and $\lambda_a = 0.1$ for the full model and all ablations, where applicable. We use a base learning rate of 2×10^{-4} , while layers of the DUS3R backbone are updated with a reduced rate of 2×10^{-5} . We also note that, like NoPoSplat, DUS3R-based splitting extensions are unable to be trained from scratch on RE10k without distillation of the weights; instead, we initialise them with MAST3R-pretrained weights, and allow other baselines to use similar supervised initialisation of the backbone where available. For a fair comparison with the baseline, we report two versions of NoPoSplat: the publicly released checkpoint (“NoPoSplat”) and a retrained variant (“NoPoSplat**”) obtained with the *exact* hyper-parameters listed above. The source code and pretrained models are publicly available at <https://github.com/m80hz/Gen2DGS>.

4.2 Results & Analysis

We analyse the relative camera pose and structure estimation performance of our splat predictions and compare it with prior art while justifying proposed loss function for training generalizable splat predictors via ablations. The section is organised

In-domain performance: We first report the in-domain performance of the proposed method and compare it with the state of the art. As Re10K dataset does not have ground truth depths, only visual comparison can be made in terms of geometric estimation. The qualitative results on sample Re10K test instances can be seen in Figure 4 and Figure 2. It can be seen that where both pose-free and pose-aware methods of the past provide noisy depthmaps that are hypersensitive to the texture – paintings on the wall, carpets, chair backs and reflections on TV screens – our approach provides much better depths (though lacks details at times, for example back of the chair). It is evident that while pixel splats depths had somewhat better coarse geometry, MVsplat and NopoSplat clearly over-fitted Gaussian parameters for the single task of view synthesis.

We use COLMAP-generated relative camera poses as used in prior art to be ground truth and compare camera poses provided by several learning based methods on Re10K testset in table 1

4.2.1 Geometry Evaluation

Some notes on geometric evaluation: When we ideally have infinite views, we can extract the surface mesh and evaluate mesh reconstruction accuracy using metrics like Chamfer distance, typically on datasets like DTU. However, when we don’t have dense multi-view coverage but instead have sufficient 2.5D RGB-D images (e.g., ScanNet), an alternative approach is to select two context views and evaluate the rendered depth. This serves as a proxy to assess interpolation/extrapolation quality on novel views—effectively measuring how well the model generalises depth consistency in a zero-shot setting.

Ideally on a dataset like DTU where 360 degree coverage of the scene is available, one can extract the ground truth surface mesh and evaluate reconstruction accuracy with a metric like Chamfer distance as done in []. As our method reconstruct scenes with two overlapping views only it is not a viable option. We have to limit the evaluation to the observed part of the scene – which is defined by union of view-frustum of input views. We thus try two alternatives: (i) mask the unobserved parts of the scene in the ground truth surface for evaluation of Chamfer distance or (ii) utilize depthmaps of the

Table 1: **Pose estimation performance (AUC) across multiple error thresholds on RE10k [10] (in-domain), ScanNet-V1 [3], and ACID [6] (both cross-domain).** Our method achieves state-of-the-art results across all thresholds and datasets, despite not being trained on ScanNet or ACID. Remarkably, our approach significantly outperforms the baseline on the cross-domain ScanNet dataset using only PnP+RANSAC, without relying on additional photometric-based pose refinement, whereas the baseline depends on such optimization. The best overall results are highlighted in **bold**, and the best-performing PnP+RANSAC only method is indicated by underline. Note that RoMa [4], in contrast to our method, is directly trained on ScanNet. **What about dust3r and mast3r why no number? What is happening in ACID dataset. Top part of the table comes from Noposplat directly... Noposplat is trained on Real0K. Write a line saying these are trained on different datasets**

		RE10k			ScanNet-V1			ACID		
	Method	5° ↑	10° ↑	20° ↑	5° ↑	10° ↑	20° ↑	5° ↑	10° ↑	20° ↑
	CoPoNeRF	0.161	0.362	0.575	-	-	-	0.078	0.216	0.398
	DUST3R	0.301	0.495	0.657	-	-	-	0.166	0.304	0.437
	MASt3R	0.372	0.561	0.709	-	-	-	0.234	0.396	0.541
	RoMa	0.546	0.698	0.797	0.168	0.361	0.575	0.463	0.588	0.689
<i>PnP+RANSAC only</i>	NoPoSplat	0.572	0.728	0.833	0.078	0.198	0.394	0.337	0.497	0.646
	Ours (2DGS)	0.588	0.737	0.832	0.085	0.223	0.432	0.344	0.513	0.659
	Ours (2DGS+Grid)	0.621	0.760	0.849	0.123	0.279	0.471	0.382	0.540	0.674
	Ours (2DGS+Normal)	0.613	0.756	0.848	0.118	0.267	0.460	0.376	0.537	0.673
	Ours (2DGS+Grid+Normal)	<u>0.627</u>	<u>0.766</u>	<u>0.855</u>	<u>0.135</u>	<u>0.289</u>	<u>0.479</u>	<u>0.392</u>	<u>0.547</u>	<u>0.679</u>
<i>w/ Photometric Optimisation</i>	NoPoSplat	0.672	0.791	0.868	0.109	0.256	0.463	0.456	0.593	0.705
	Ours (2DGS)	0.672	0.788	0.859	0.129	0.298	0.515	0.460	0.599	0.713
	Ours (2DGS+Grid)	0.686	0.799	0.870	0.136	0.311	0.512	0.474	0.607	0.718
	Ours (2DGS+Normal)	0.679	0.798	0.871	0.141	0.323	0.520	0.475	0.610	0.721
	Ours (2DGS+Grid+Normal)	0.689	0.804	0.876	0.156	0.334	0.539	0.488	0.619	0.726

Table 2: **Pose estimation performance (AUC) across multiple error thresholds on ScanNet-1500 [3] (cross-domain).** Our method achieves state-of-the-art results across all thresholds despite not being trained on ScanNet. The best overall results are highlighted in **bold**, and the best-performing PnP+RANSAC only method is indicated by underline. Note that RoMa [4], in contrast to our method, is directly trained on ScanNet. **What is special about this table? why scannet 1500? You have no improvement over Nopo so why talk about these things? Remove this**

		ScanNet-1500		
	Method	5° ↑	10° ↑	20° ↑
	DUST3R	0.221	0.437	0.636
	MASt3R	0.159	0.359	0.573
	RoMa	0.271	0.476	0.653
<i>PnP+RANSAC only</i>	NoPoSplat	0.232	0.443	0.634
	Ours (2DGS)	0.237	0.451	0.637
	Ours (2DGS+Grid)	0.238	0.467	0.657
	Ours (2DGS+Normal)	0.242	0.471	0.659
	Ours (2DGS+Grid+Normal)	<u>0.250</u>	<u>0.476</u>	<u>0.662</u>
<i>w/ Photometric Optimisation</i>	NoPoSplat	0.262	0.474	0.651
	Ours (2DGS)	0.241	0.462	0.650
	Ours (2DGS+Grid)	0.261	0.473	0.658
	Ours (2DGS+Normal)	0.255	0.480	0.657
	Ours (2DGS+Grid+Normal)	0.263	0.479	0.666

observed views within the union of context view-frustum and report average depth error (2.5D) on all the test views as a proxy for 3D geometric accuracy of estimated splats.

Depth Evaluation. Quantitative, Qualitative, ...

Pose Evaluation. Quantitative, Qualitative, ...

Table 3: **Depth estimation performance for novel views on ScanNet-V1 [3].** Two input camera views are used to reconstruct the scene using Gaussians. We propose using the accuracy of rendered depth for novel views as a holistic measure of the model’s interpolation capability for 3D reconstruction. Our method outperforms all other approaches across all evaluated metrics by clear margin. The best overall results are highlighted in **bold**, and the best-performing method without camera pose refinement is indicated by underline. **What other methods should be here**

		Rendered Depth (Novel views)		
Method		Abs Rel↓	$\delta_1 < 1.10 \uparrow$	$\delta_1 < 1.25 \uparrow$
	pixelSplat	0.299	0.552	0.818
	MVSplat	0.189	0.412	0.745
<i>w/o Pose Refine.</i>	NoPoSplat	0.131	0.554	0.851
	NoPoSplat*	0.135	0.552	0.847
	Ours (2DGS)	0.121	0.668	0.879
	Ours (2DGS+Grid)	0.109	0.679	0.890
	Ours (2DGS+Normal)	0.115	0.674	0.884
	Ours (2DGS+Grid+Normal)	<u>0.108</u>	<u>0.680</u>	<u>0.892</u>
<i>w Pose Refine.</i>	NoPoSplat	0.126	0.567	0.861
	NoPoSplat*	0.130	0.564	0.856
	Ours (2DGS)	0.114	0.692	0.884
	Ours (2DGS+Grid)	0.102	0.706	0.901
	Ours (2DGS+Normal)	0.106	0.704	0.898
	Ours (2DGS+Grid+Normal)	0.100	0.707	0.904

Table 4: **Depth estimation performance for input views on ScanNet-V1 [3].** The scene is reconstructed from two input camera views using Gaussians, and the reported “rendered depth” is computed directly from these views. The first input view serves as the global reference frame. The best overall results are highlighted in **bold**, and the best-performing method without camera pose refinement is indicated by underline. **see comment, we should remove the depth 1 from here. Should we not compare it with supervised and unsupervised depth?**

		Input 1 Rendered Depth			Input 2 Rendered Depth		
Method		Abs Rel↓	$\delta_1 < 1.10 \uparrow$	$\delta_1 < 1.25 \uparrow$	Abs Rel↓	$\delta_1 < 1.10 \uparrow$	$\delta_1 < 1.25 \uparrow$
<i>w/o Pose Refine.</i>	NoPoSplat	0.127	0.575	0.864	0.138	0.530	0.833
	NoPoSplat*	0.129	0.573	0.864	0.139	0.529	0.834
	Ours (2DGS)	0.110	0.701	0.892	0.127	0.628	0.858
	Ours (2DGS+Grid)	0.103	0.710	0.904	0.119	0.633	0.868
	Ours (2DGS+Normal)	0.105	0.709	0.898	0.123	0.632	0.863
	Ours (2DGS+Grid+Normal)	<u>0.101</u>	<u>0.713</u>	<u>0.906</u>	<u>0.118</u>	<u>0.637</u>	<u>0.870</u>
<i>w Pose Refine.</i>	NoPoSplat	0.127	0.575	0.864	0.122	0.572	0.865
	NoPoSplat*	0.129	0.573	0.864	0.123	0.569	0.865
	Ours (2DGS)	0.110	0.701	0.892	0.100	0.709	0.896
	Ours (2DGS+Grid)	0.103	0.710	0.904	0.097	0.717	0.903
	Ours (2DGS+Normal)	0.105	0.709	0.898	0.099	0.716	0.902
	Ours (2DGS+Grid+Normal)	0.101	0.713	0.906	0.095	0.719	0.908

4.2.2 Novel View Synthesis Evaluation

4.3 Ablations Studies

...

Table 5: **Novel-view synthesis performance on the RealEstate10k dataset [10].** We compare pose-required and pose-free methods against our proposed variants. “NoPoSplat” refers to results from publicly available checkpoints, while “NoPoSplat*” indicates results from our retrained baseline using identical hyperparameters for fair comparison. The best results among pose-free methods (and without optimizing camera poses for target views) are highlighted in **bold**, and the best-performing pose-required method is indicated by underline.

		Small			Medium			Large			Average		
Method		PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Pose-Required	pixelNeRF	18.417	0.601	0.526	19.930	0.632	0.480	20.869	0.639	0.458	19.824	0.626	0.485
	AttnRend	19.151	0.663	0.368	22.532	0.763	0.269	25.897	0.845	0.186	22.664	0.762	0.269
	pixelSplat	20.263	0.717	0.266	23.711	0.809	0.181	27.151	0.879	0.122	23.848	0.806	0.185
	MVSplat	<u>20.353</u>	<u>0.724</u>	<u>0.250</u>	<u>23.778</u>	<u>0.812</u>	<u>0.173</u>	<u>27.408</u>	<u>0.884</u>	<u>0.116</u>	<u>23.977</u>	<u>0.811</u>	<u>0.176</u>
Pose-Free	DUS3R	14.101	0.432	0.468	15.419	0.451	0.432	16.427	0.453	0.402	15.382	0.447	0.432
	MASt3R	13.534	0.407	0.494	14.945	0.436	0.451	16.028	0.444	0.418	14.907	0.431	0.452
	Splatt3R	14.352	0.475	0.472	15.529	0.502	0.425	15.817	0.483	0.421	15.318	0.490	0.436
	CoPoNeRF	17.393	0.585	0.462	18.813	0.616	0.392	20.464	0.652	0.318	18.938	0.619	0.388
	NoPoSplat	21.086	0.721	0.237	23.134	0.776	0.185	25.086	0.818	0.141	23.189	0.775	0.185
	NoPoSplat*	21.097	0.723	0.237	23.191	0.779	0.187	25.107	0.817	0.144	23.244	0.778	0.187
	Ours (2DGS)	21.051	0.725	0.245	23.338	0.784	0.187	25.541	0.830	0.140	23.407	0.782	0.188
	Ours (2DGS+Grid)	21.341	0.736	0.236	23.453	0.788	0.185	25.529	0.829	0.141	23.528	0.787	0.185
	Ours (2DGS+Normal)	21.275	0.733	0.236	23.198	0.780	0.187	25.205	0.823	0.144	23.300	0.781	0.187
	Ours (2DGS+Grid+Normal)	21.349	0.739	0.234	23.463	0.789	0.184	25.628	0.832	0.141	23.564	0.789	0.184

Table 6: **Novel-view synthesis performance on the RealEstate10k dataset [10].** We compare pose-free methods against our proposed variants. For completeness, this table reports results obtained by optimizing camera poses for target views, although incorporating target-view information may not be fully justified for a genuine novel-view synthesis evaluation, despite being practiced in some prior works. “NoPoSplat” refers to results obtained from publicly available checkpoints, while “NoPoSplat*” indicates results from our retrained baseline using identical hyper-parameters for fair comparison. The overall best results among all methods are highlighted in **bold**.

		Small			Medium			Large			Average		
Method		PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
<i>Pose-Free w/ Align.</i>	NoPoSplat	22.514	0.784	0.210	24.899	0.839	0.160	27.411	0.883	0.119	25.033	0.838	0.160
	NoPoSplat*	22.533	0.785	0.211	24.905	0.839	0.161	27.419	0.884	0.120	25.036	0.839	0.161
	Ours (2DGS)	22.211	0.774	0.224	24.715	0.835	0.166	27.209	0.880	0.124	24.813	0.833	0.168
	Ours (2DGS+Grid)	22.376	0.780	0.217	24.733	0.836	0.166	27.228	0.880	0.125	24.869	0.835	0.167
	Ours (2DGS+Normal)	22.432	0.784	0.214	24.757	0.837	0.165	27.136	0.879	0.125	24.867	0.836	0.166
	Ours (2DGS+Grid+Normal)	22.423	0.784	0.215	24.787	0.838	0.165	27.282	0.882	0.124	24.941	0.838	0.165

5 Conclusion

Finally ...

References

- [1] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *CVPR*, 2024. 7
- [2] Yuedong Chen, Haofei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. Mvsplat: Efficient 3d gaussian splatting from sparse multi-view images. *arXiv preprint arXiv:2403.14627*, 2024. 3, 7
- [3] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *CVPR*, 2017. 3, 7, 9, 10, 12, 14
- [4] Johan Edstedt, Qiyu Sun, Georg Bökman, Márten Wadenbäck, and Michael Felsberg. Roma: Robust dense feature matching. In *CVPR*, 2024. 3, 9
- [5] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM TOG*, 2023. 1

Table 7: **Zero-shot out-of-distribution novel-view synthesis performance on ACID [6] and ScanNet-V1 [3], compared against state-of-the-art methods.** All models are trained exclusively on the RE10k dataset. The overall best results are highlighted in **bold**. Note that these results are obtained without optimizing camera poses for target views. *Should we be focusing on these results at all? hardly any difference form the Nopo on scannet and minimal impact on ACID!*

Method	ACID			ScanNet-V1		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NoPoSplat	23.379	0.683	0.238	21.068	0.646	0.268
<i>Pose-Free</i> Ours (2DGS)	23.914	0.704	0.233	21.167	0.648	0.267
Ours (2DGS+Grid)	23.745	0.697	0.235	20.919	0.644	0.271
Ours (2DGS+Normal)	23.848	0.702	0.235	21.178	0.648	0.271
Ours (2DGS+Grid+Normal)	23.950	0.707	0.233	21.163	0.648	0.268

Table 8: **Zero-shot out-of-distribution novel-view synthesis performance on ACID [6] and ScanNet-V1 [3], compared against state-of-the-art methods.** All models are trained exclusively on the RE10k dataset. For completeness, this table reports results obtained by optimizing camera poses for target views, although incorporating target-view information may not be fully justified for a genuine novel-view synthesis evaluation, despite its use in some prior works. The overall best results are highlighted in **bold**.

Method	ACID			ScanNet-V1		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NoPoSplat	25.765	0.776	0.198	29.002	0.876	0.145
<i>Pose-Free</i> Ours (2DGS)	25.627	0.773	0.202	28.728	0.868	0.145
Ours (2DGS+Grid)	25.629	0.772	0.202	28.575	0.864	0.148
<i>Align.</i> Ours (2DGS+Normal)	25.618	0.772	0.203	28.769	0.869	0.149
Ours (2DGS+Grid+Normal)	25.734	0.776	0.201	28.984	0.873	0.145

- [6] Andrew Liu, Richard Tucker, Varun Jampani, Ameesh Makadia, Noah Snavely, and Angjoo Kanazawa. Infinite nature: Perpetual view generation of natural scenes from a single image. In *ICCV*, 2021. 3, 7, 9, 12, 14
- [7] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *CVPR*, 2016. 7
- [8] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *CVPR*, 2024. 3, 4, 5
- [9] Botao Ye, Sifei Liu, Haofei Xu, Li Xueting, Marc Pollefeys, Ming-Hsuan Yang, and Peng Songyou. No pose, no problem: Surprisingly simple 3d gaussian splats from sparse unposed images. In *ICLR*, 2025. 3, 4, 7, 8
- [10] Tinghui Zhou, Richard Tucker, John Flynn, Graham Fyffe, and Noah Snavely. Stereo magnification: learning view synthesis using multiplane images. *ACM TOG*, 2018. 3, 7, 9, 11, 14

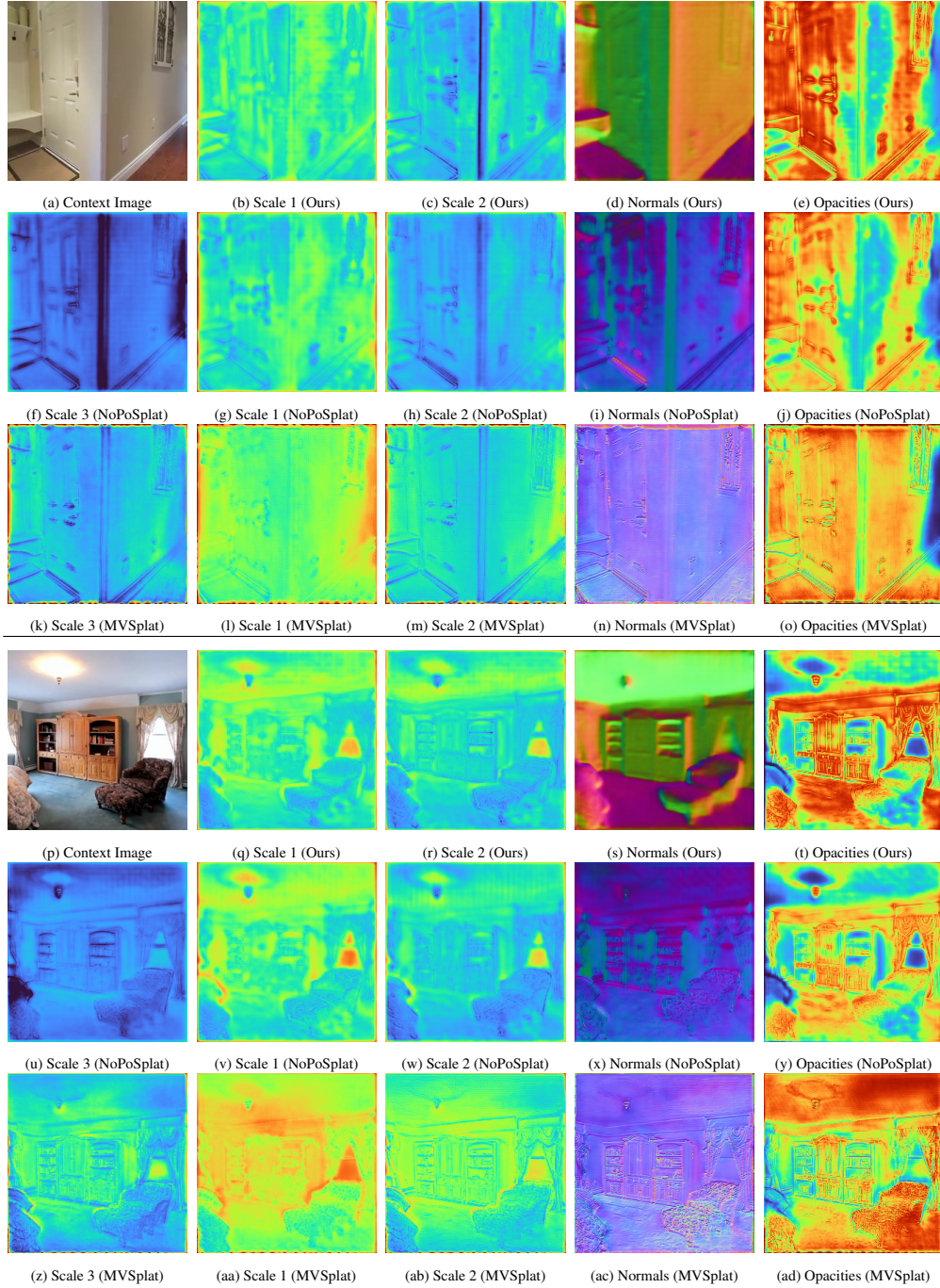


Figure 2: Qualitative comparison of Gaussians. The first row visualises our 2D Gaussians with the second and third containing results from NoPoSplat and MVsplat, respectively. All visualised Gaussians correspond to the context image (a). Row 1: (b-c) shows the extent of 2D Gaussians along the estimated scene surface it belongs to. The surface is defined by its normal, which is shown in (d). Row 2/3 for NoPoSplat and MVsplat respectively: (f/k) show minimal Gaussian elongation. This direction is assumed to be perpendicular to the dominant surface and the elongation quantises the uncertainty in the surface localisation. (g/l, h/m) show Gaussian elongation along the estimated dominant surface, i.e. comparable to corresponding columns in top row, (i/n) in the two rows shows estimated normals of the dominant surface for baseline methods. Additionally, (e,j,o) show the estimated opacity of the Gaussians. textcolor magentaAdd ablation figures for scene and normals as well

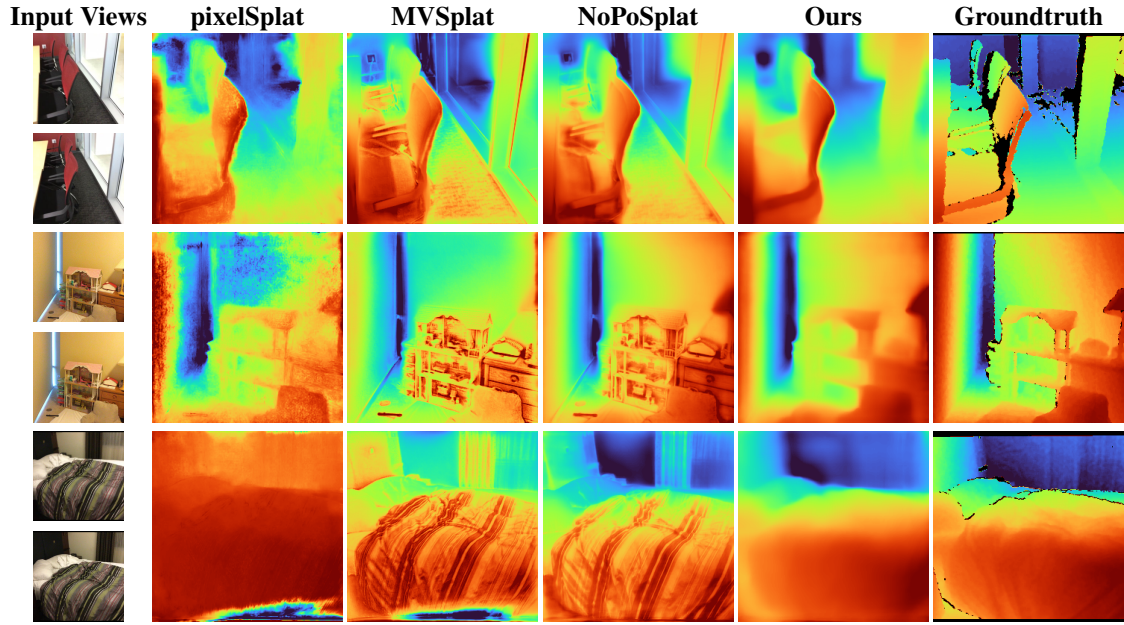


Figure 3: **Qualitative comparison of rendered novel-view depth on ScanNet-V1 [3].**

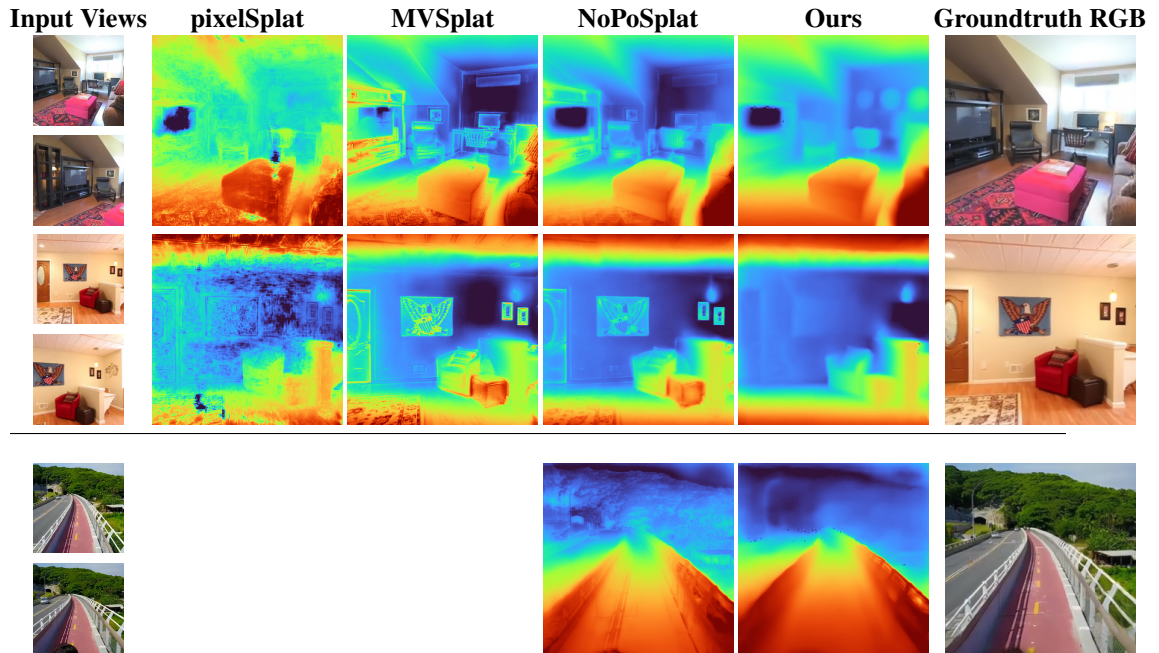


Figure 4: **Qualitative comparison of rendered novel-view depth on RE10k [10] (top two rows) and ACID [6] (bottom row).** *When will this finish?*